Robotivity logo

Materialy szkoleniowe — UiPath Dzien 3

Autoryzowany Partner UiPath

info@robotivity.pl · www.Robotivity.pl · tel.: +48 690 020 576

DZIEN 3 — PLIKI I FOLDERY, FLOWCHART, ORCHESTRATOR, STATE MACHINE

Modul 1 — Pliki i foldery

OPERACJE NA PLIKACH Z PERSPEKTYWY RPA

Robot RPA operuje na plikach i folderach dokladnie tak, jak uzytkownik komputera — kopiuje, przenosi, tworzy i usuwa. Roznica polega na tym, ze robot nie widzi eksploratora plikow — musi znac dokladne sciezki i poslugiwac sie dedykowanymi aktywnosciami.

Najwazniejsze zasady pracy z plikami w RPA

- **Nigdy nie hardkoduj sciezek bezwzglednych** (np. `C:\Users\Jan\Dokumenty\...`) — zamiast tego uzywaj `Directory.GetCurrentDirectory`, `Path.Combine()` lub assetow z Orchestratora. Robot moze dzialac na innej maszynie niz ta, na ktorej go programujesz.
- **Zawsze weryfikuj istnienie pliku/folderu** przed operacja — uzywaj `Path Exists`, aby uniknac wyjatkow w runtime.
- **Pamietaj o blokadach plikow** — jesli plik Excel jest otwarty w tle (przez uzytkownika lub inny proces), robot nie bedzie mogl go odczytac ani zapisac. Dlatego dobra praktyka to `Kill Process "excel"` na poczatku procesu.
- **Uprawnienia** — robot dziala w kontekscie konta uzytkownika (lub konta serwisowego). Jesli konto nie ma uprawnien do danego folderu, robot dostanie wyjatek `UnauthorizedAccessException`.

FILEINFO I DIRECTORYINFO — DLACZEGO NIE STRING?

W UiPath aktywnosc `For Each File in Folder` zwraca zmienna `CurrentFile` typu `FileInfo` (nie `String`). To typ z biblioteki .NET (`System.IO.FileInfo`), ktory jest "inteligentnym opakowaniem" na sciezke pliku.

Dlaczego to wazne

- **String** to po prostu tekst — np. `"C:\Dane\Raport.xlsx"`. Zeby wyciagnac z niego nazwe pliku, musisz recznie parsowac tekst.
- **FileInfo** to obiekt, ktory "zna" swoj plik — udostepnia gotowe wlasciwosci: Name, Extension, FullName, Folder, SizeInKB, CreationTime, LastWriteTime.
- **DirectoryInfo** to analogiczny obiekt dla folderow — udostepnia: FullName, Name, Parent, CreationTime.

> **Wskazowka:** W praktyce uzywaj wlasciwosci `CurrentFile.*` zamiast recznego parsowania sciezek — jest bezpieczniej i czytelniej.

Wlasciwosci FileInfo

Wlasciwosc (FileInfo)	Opis	Przyklad
`CurrentFile.Name`	Pelna nazwa pliku z rozszerzeniem	`Raport.xlsx`
`CurrentFile.FullName`	Pelna sciezka	`C:\Dane\Raport.xlsx`
`CurrentFile.Extension`	Rozszerzenie	`.xlsx`
`CurrentFile.Folder`	Sciezka katalogu	`C:\Dane`
`CurrentFile.SizeInKB`	Rozmiar w KB	`12.5`
`CurrentFile.CreationTime`	Data utworzenia	`2024-01-15 10:30:00`
`CurrentFile.LastWriteTime`	Data ostatniej modyfikacji	`2024-06-20 14:22:00`

PATH.COMBINE — DLACZEGO NIE LACZYC SCIEZEK RECZNIE?

Typowy blad poczatkujacych

```
str_sciezka = str_folder + "\" + str_plik
```

Problem: co jesli `str_folder` juz konczy sie na `\`? Wtedy dostaniesz podwojny separator: `C:\Dane\\Raport.xlsx`. Albo co jesli pracujesz na systemie, ktory uzywa `/` (Linux/Mac)?

> **Rozwiazanie:** `Path.Combine(str_folder, str_plik)` — automatycznie dodaje separator, tylko gdy jest potrzebny, i uzywa odpowiedniego separatora dla systemu operacyjnego.

```
Path.Combine("C:\Dane", "Raport.xlsx")    → "C:\Dane\Raport.xlsx"
Path.Combine("C:\Dane\", "Raport.xlsx")   → "C:\Dane\Raport.xlsx"  (brak podwojnego \)
```

KLUCZOWE AKTYWNOSCI

Aktywnosc	Opis	Wazne wlasciwosci
For Each File in Folder	Iteruje po plikach w folderze	Filter (np. `*.xlsx`), IncludeSubDirectories, OrderBy
Copy File	Kopiuje plik do wskazanej lokalizacji	Path, Destination, Overwrite
Move File	Przenosi plik (wycina + wkleja)	Path, Destination
Path Exists	Sprawdza czy plik/folder istnieje	Path, PathType (File/Folder), zwraca Boolean
Create Folder	Tworzy nowy katalog	Path, Output → ILocalResource
Extract/Unzip Files	Rozpakowuje archiwum ZIP	FileToExtract, ExtractToADedicatedFolder, DestinationFolderInfo → DirectoryInfo
Select Folder	Okno dialogowe wyboru folderu	SelectedFolder → String
Get Environment Variable	Pobiera zmienna srodowiskowa	Variable=CurrentDirectory → biezacy katalog projektu

Dodatkowe aktywnosci

- **Build Data Table** — tworzy pusta tabele z predefiniowanymi kolumnami (edytor wizualny). Przydatne gdy chcemy recznie budowac raport.
- **Modify Text** — aktywnosc do manipulacji tekstem w pipeline (Find and Replace, ToUpper/ToLower). Alternatywa dla wyrazen VB.NET.
- **Write CSV** — zapisuje DataTable do pliku CSV. Wlasciwosci: FilePath, DataTable, Delimitator, AddHeaders.
- **Switch** — odpowiednik switch/case. Pozwala zdefiniowac wiele sciezek na podstawie jednej wartosci. Typ generyczny (np. `Switch<String>`). Posiada sekcje Default dla nieobsluzonych wartosci.

Wyrazenia VB.NET przydatne w module

```
Path.Combine(folder, "plik.txt")     ' laczy sciezki bezpiecznie
Directory.GetCurrentDirectory          ' biezacy katalog robota
CurrentFile.Extension.Replace(".",Nothing).ToUpper  ' rozszerzenie bez kropki, wielkie litery
```

Cwiczenia — Modul 1

Zadanie 1 — Kopiowanie pliku

Cel: Skopiuj plik Produkty.xlsx do folderu wybranego przez uzytkownika i sprawdz czy kopiowanie sie powiodlo.

CZEGO SIE NAUCZYSZ

- `Get Environment Variable` z opcja `CurrentDirectory` — alternatywa dla hardkodowania sciezek
- `Select Folder` — interakcja z uzytkownikiem (okno dialogowe systemowe)
- `Path.Combine` — bezpieczne laczenie sciezek
- `Path Exists` — walidacja operacji plikowych

Zadanie 2 — Rozpakowanie ZIP i raport CSV

Cel: Rozpakuj Raporty.zip, zbierz szczegoly plikow (nazwa, folder, rozszerzenie, rozmiar) do DataTable i zapisz jako CSV.

CZEGO SIE NAUCZYSZ

- `Extract/Unzip Files` — rozpakowywanie archiwow z opcja `ExtractToADedicatedFolder`
- `DirectoryInfo` vs `String` — DirectoryInfo daje dostep do metadanych folderu
- `Modify Text` — pipeline do transformacji tekstu (Find&Replace + ToUpper)
- `Build Data Table` + `Add Data Row` — reczne budowanie tabel
- `Write CSV` — eksport danych do formatu CSV

Zadanie 3 — Sortowanie plikow wg rozszerzenia

Cel: W katalogu z plikami stworz foldery PDF, DOCX, XLSX i przeniesc pliki do odpowiednich folderow.

CZEGO SIE NAUCZYSZ

- `Array of String` — inicjalizacja: `{"PDF", "DOCX", "XLSX"}`
- `Switch` — wielogaleziowe rozgalezienie (lepsze niz zagniezdzone If/Else)
- `Move File` vs `Copy File` — Move przenosi (usuwa zrodlo), Copy zostawia oryginal
- `ILocalResource` — typ zwracany przez `Create Folder`, ma metode `.GetName`
- Default w Switch — sekcja dla nieobsluzonych przypadkow

Modul 2 — Flowchart: Biala Lista (CEIDG)

TEORIA — FLOWCHART

Czym jest Flowchart w UiPath?

Flowchart to drugi (obok Sequence) podstawowy typ kontenera w UiPath. O ile Sequence wykonuje aktywnosci liniowo — jedna po drugiej, z gory na dol — o tyle Flowchart pozwala na swobodne laczenie elementow strzalkami w dowolnych kierunkach.

Flowchart najlepiej sprawdza sie, gdy:

- Proces ma **wiele rozgalezien** (wiecej niz 2-3 sciezki)
- Potrzebujesz **petli z wieloma warunkami wyjscia** (np. przetwórz dane, ale zakoncz jesli blad LUB brak danych LUB timeout)
- Chcesz **zwizualizowac przeplyw** procesu w sposob zblizony do klasycznego diagramu blokowego
- Proces ma **nielinearny przebieg** — np. po bledzie wraca do wczesniejszego kroku

Kiedy NIE uzywac Flowchart:

- Dla prostych, liniowych procesow — tu Sequence jest czytelniejszy
- Gdy nie potrzebujesz petli ani rozgalezien — Flowchart dodaje niepotrzebna zlozonosc wizualna

FLOWCHART VS SEQUENCE — POROWNANIE

Cecha	Sequence	Flowchart
Przebieg	Liniowy, z gory na dol	Swobodny, z polaczeniami
Rozgalezienia	If/Else, Switch	FlowDecision, FlowSwitch
Petle	For Each, While	Polaczenia wsteczne (strzalki)
Kiedy uzywac	Proste, sekwencyjne procesy	Procesy z wieloma sciezkami, petlami, decyzjami
Widocznosc	Kompaktowy	Przejrzysty dla zlozonej logiki

ELEMENTY FLOWCHART

- **FlowStep** — pojedynczy krok (zawiera jedna aktywnosc lub Sequence). Moze miec jedno wyjscie (strzalke do nastepnego kroku).
- **FlowDecision** — odpowiednik If/Else. Przyjmuje warunek Boolean i ma **dwie galezi**: True (zielona strzalka) i False (czerwona strzalka).
- **FlowSwitch** — odpowiednik Switch. Pozwala na **wiele galezi** na podstawie wartosci wyrazenia.
- **Strzalki (connections)** — lacza elementy i definiuja przeplyw. Strzalka wsteczna (do wczesniejszego elementu) tworzy petle.

> **Petla w Flowchart** — realizowana przez polaczenie strzalki z FlowStep z powrotem do wczesniejszego FlowStep lub FlowDecision. Nie wymaga dedykowanej aktywnosci petli — wystarczy narysowac strzalke "wstecz".

Zasady projektowania czytelnych Flowchartow

1. **Przeplyw z gory na dol** — Start na gorze, End na dole
2. **Decyzje wyrownane** — True w jedna strone, False w druga (konsekwentnie)
3. **Grupuj logike w Sequence** — wewnatrz FlowStep umiesc Sequence z wieloma aktywnosciami
4. **Nazywaj FlowDecision** — nadawaj czytelne nazwy (np. "Czy plik istnieje?")
5. **Unikaj krzyzujacych sie strzalek** — przesun elementy, aby strzalki nie przecinaly sie

Cwiczenia — Modul 2

Cwiczenie — Weryfikacja NIP na Bialej Liscie

Cel: Na podstawie numerow NIP z pliku CEIDG.xlsx sprawdz na stronie podatki.gov.pl, czy firma figuruje w rejestrze VAT. Zapisz wyniki do nowej kolumny w Excelu.

CZEGO SIE NAUCZYSZ

- Flowchart z petla — strzalka powrotna zamiast For Each
- FlowDecision jako warunek wyjscia z petli
- `dt.Rows(index)("kolumna").ToString` — dostep do konkretnej komorki DataTable
- Indeksowanie od 0 — `Rows(int_licznik - 1)` bo licznik startuje od 1
- `Use Application/Browser` z CloseMode — Never (w trakcie pracy) vs Always (na koncu)
- **Check App State** — sprawdzenie czy element pojawil sie na stronie
- **Zapis wyniku do DataTable** — `dt.Rows(index)("kolumna") = wartosc` modyfikuje tabele w pamieci

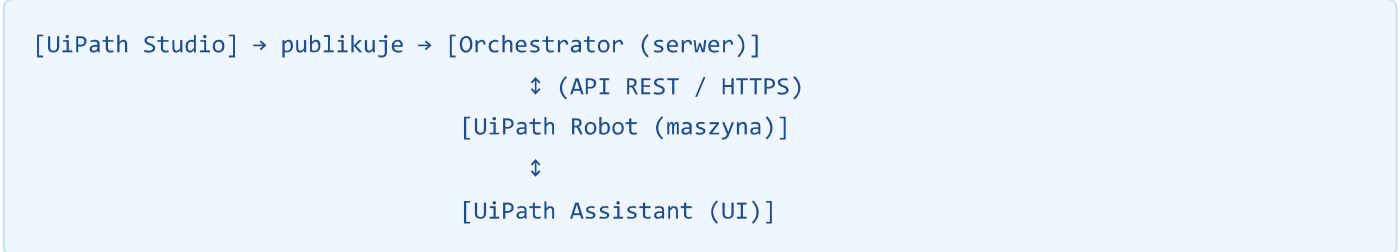
Modul 3 — Orchestrator (Assets, Queues, Dispatcher/Performer)

CZYM JEST ORCHESTRATOR?

Orchestrator to serwer zarzadzajacy robotami UiPath. Pelni trzy glowne role:

1. **Zarzadzanie robotami** — Orchestrator wie, ktore roboty (maszyny) sa dostepne, i moze zlecac im uruchomienie procesow. Robot laczy sie z Orchestratorem przez UiPath Assistant.
2. **Centralne przechowywanie danych** — Assety (dane konfiguracyjne, hasla) i Kolejki (dane do przetworzenia) sa przechowywane na serwerze, a nie na maszynie robota. To oznacza, ze mozna zmienic haslo w jednym miejscu i wszystkie roboty od razu uzyja nowego.
3. **Monitoring i logowanie** — Orchestrator zbiera logi z wszystkich robotow, pokazuje statusy transakcji, generuje raporty i alerty.

Architektura komunikacji Robot - Orchestrator



- Robot laczy sie z Orchestratorem przez HTTPS (szyfrowane polaczenie)
- Cala komunikacja odbywa sie przez REST API

- W trybie "Attended" (robot z uzytkownikiem) — robot dziala na maszynie uzytkownika
- W trybie "Unattended" (robot bez uzytkownika) — robot dziala na dedykowanej maszynie wirtualnej

ASSETY

Typ Asseta	Opis	Uzycie
Text	Przechowuje tekst (np. sciezka do aplikacji)	Get Asset → String
Integer	Przechowuje liczbe	Get Asset → Int32
Boolean	Przechowuje True/False	Get Asset → Boolean
Credential	Przechowuje login + haslo (SecureString)	Get Credential → Username (String) + Password (SecureString)

Aktywnosci

- **Get Asset** — pobiera wartosc asseta tekstowego/liczbowego/boolean
- **Get Credential** — pobiera login i haslo z asseta typu Credential
- Wlasciwosc `FolderPath` — folder Orchestratora (np. `"user@mail.com's workspace"`)

KOLEJKI (QUEUES)

Kolejka to centralny "schowek" na elementy do przetworzenia. Dziala jak tasma produkcyjna w fabryce — jeden robot (Dispatcher) kladzie elementy na tasme, a drugi (Performer) zdejmuje je i przetwarza.

Cykl zycia Queue Item

```
[New] → Robot pobiera → [In Progress] → Robot konczy → [Successful]
                                      → Robot zglasza blad → [Failed]
                                           ├── Business Exception (blad danych)
                                           └── Application Exception (blad systemowy)
```

Pojecie	Opis
Queue	Kolejka w Orchestratorze — kontener na elementy do przetworzenia
Queue Item	Pojedynczy element kolejki z danymi (SpecificContent)
Reference	Unikalna sygnatura/identyfikator transakcji
Status	New, In Progress, Successful, Failed
Error Type	Business (blad danych) vs Application (blad systemowy)

Kiedy Queue, a kiedy DataTable?

Kryterium	Queue (Orchestrator)	DataTable (plik Excel)
Wiele robotow	Tak — kolejka synchronizuje dostep	Nie — plik blokowany
Retry automatyczny	Tak — konfigurowalny w Orchestratorze	Nie — musisz sam zaprogramowac
Status tracking	Wbudowany (New/InProgress/Successful/Failed)	Musisz sam dodac kolumne "Status"
Audyt	Pelny log w Orchestratorze	Brak
Offline	Nie — wymaga polaczenia z Orchestratorem	Tak — dziala lokalnie
Prostota	Wymaga konfiguracji Orchestratora	Prosty Read Range

> **Zasada:** Jesli masz jednego robota i proste dane — DataTable wystarczy. Jesli masz wiele robotow, potrzebujesz retry lub audytu — uzyj Queue.

AKTYWNOSCI KOLEJEK

Aktywnosc	Opis
Add Queue Item	Dodaje jeden element z ItemInformation (klucz-wartosc)
Bulk Add Queue Items	Dodaje wiele elementow hurtowo z DataTable
Get Transaction Item	Pobiera nastepny element ze statusem New
Set Transaction Status	Ustawia status: Successful / Failed + ErrorType + Reason

WZORZEC DISPATCHER-PERFORMER

- **Dispatcher** — robot ladujacy dane do kolejki (np. z Excela)
- **Performer** — robot przetwarzajacy elementy z kolejki jeden po jednym

Wazne: Add Queue Item pozwala nadac unikalne Reference (sygnature). Bulk Add Queue Items jest szybszy, ale nie pozwala nadac indywidualnych referencji.

TRY CATCH VS RETRY SCOPE — KIEDY CO UZYWAC

Cecha	Try Catch	Retry Scope
Cel	Przechwycenie bledu i reakcja	Automatyczne ponawianie do sukcesu
Ile razy?	Raz (catch)	Konfigurowalnie (RetryNumber)
Warunek sukcesu	Brak wyjatku	Spelnienie warunku (Condition)
Kiedy uzywac	Obsluga znanych bledow, logowanie, fallback	Element UI laduje sie wolno, niestabilna siec
Przyklad	Try: Get Transaction Item, Catch: Assign Nothing	Retry 3x: Click "Login" → Condition: Element "Dashboard" Exists

Wskazowka: Try Catch i Retry Scope mozna laczyc — Retry Scope wewnatrz Try, aby ponawiac operacje, a jesli wszystkie proby zawioda, Try Catch zlapie wyjatek.

LOGI ORCHESTRATORA

Orchestrator automatycznie zbiera wszystkie logi generowane przez `Log Message` i wyswietla je w panelu **Logs** (Monitoring → Logs). Dlatego:

- W produkcji uzywamy **Log Message** zamiast **Message Box** — Message Box blokuje robota i nie trafia do Orchestratora
- Poziomy logow (Trace/Info/Warn/Error) pomagaja filtrowac problemy w panelu Orchestratora
- Kazdy log zawiera: timestamp, poziom, nazwe robota, nazwe procesu, tresc wiadomosci

Cwiczenia — Modul 3

Zadanie 1 — Logowanie z Orchestratora

Cel: Pobierz dane logowania i sciezke do aplikacji z Orchestratora, uruchom Robotivity i zaloguj sie.

CZEGO SIE NAUCZYSZ

- `Get Credential` — pobieranie loginu i hasla z asseta Credential
- `Get Asset` — pobieranie sciezki do aplikacji z asseta Text
- `SecureString` — haslo nigdy nie jest widoczne w logach, uzywamy SecureText w Type Into
- `Check App State` — weryfikacja czy logowanie sie powiodlo
- `Kill Process` na poczatku — dobre praktyki: wyczysc srodowisko przed startem

Uwaga: W `Type Into` dla hasla uzywamy wlasciwosci `SecureText` (nie `Text`!) — dzieki temu haslo nigdy nie zostanie wyswietlone w logach.

Zadanie 2 — Dispatcher

Cel: Zaladuj dane z Transactions.xlsx do kolejki w Orchestratorze.

CZEGO SIE NAUCZYSZ

- `Bulk Add Queue Items` — szybkie ladowanie danych z DataTable do kolejki
- `Add Queue Item` — ladowanie z unikalnymi referencjami (wolniejsze, ale z kontrola)
- Trade-off: szybkosc (Bulk) vs kontrola (Add z referencjami)
- `CurrentRow.Item(0).ToString` — dostep do kolumny po indeksie; bezpieczniej: `CurrentRow("CashIn").ToString` (po nazwie)

Wazne: Jesli dodajemy indywidualny numer referencji, scope zmiennej licznika musi byc na wyzszym poziomie niz petla For Each Row — w przeciwnym razie wartosc resettuje sie w kazdej iteracji.

Zadanie 3 — Performer

Cel: Stworz robota (Flowchart), ktory pobiera transakcje z kolejki, loguje sie do Robotivity i wpisuje dane. Obsluga bledow: zle haslo → blad systemowy, dane nienumeryczne → Failed (Business).

CZEGO SIE NAUCZYSZ

- Flowchart z petla — strzalka powrotna do "Pobranie transakcji" po procesowaniu
- `Get Transaction Item` — automatycznie pobiera element ze statusem New i zmienia na InProgress
- Try Catch wokol `Get Transaction Item` — gdy kolejka pusta, rzuca wyjatek
- `TransactionItem IsNot Nothing` — warunek wyjscia z petli
- `.SpecificContent.GetValueOrDefault("klucz")` — bezpieczny dostep do danych transakcji (zwraca `Object` — potrzebny `.ToString`!)
- `.IsNumeric` — metoda VB.NET sprawdzajaca czy tekst jest liczba
- `Set Transaction Status` — Successful vs Failed (Business) — roznica w raportowaniu

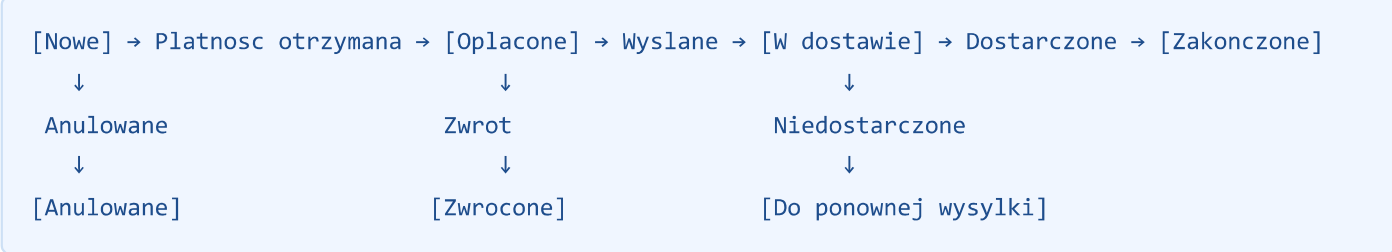
Uwaga: W przypadku zlego hasla robot ma zakonczyc prace z komunikatem bledu systemowego. W przypadku danych nienumerycznych — zmienia status transakcji na Failed z ErrorType: Business.

Modul 4 — State Machine: wprowadzenie

CZYM JEST STATE MACHINE?

Automat stanowy (State Machine) to wzorzec projektowy, w ktorym proces przechodzi miedzy zdefiniowanymi stanami (States) na podstawie warunkow (Transitions). To jak mapa stanow — w kazdym momencie proces "jest" w dokladnie jednym stanie i na podstawie warunkow przechodzi do innego.

Przyklad ze swiata biznesu — proces zamowienia



Kazdy prostokat to **stan** (State), a strzalki to **przejscia** (Transitions) z warunkami.

ELEMENTY STATE MACHINE

Element	Opis
State	Stan — zawiera Entry (akcje przy wejsciu), Exit (akcje przy wyjsciu) i Transitions
Transition	Przejscie miedzy stanami — zawiera Condition (warunek) i wskazanie docelowego State
Initial State	Stan poczatkowy (jeden, oznaczony strzalka startowa)
Final State	Stan koncowy — brak Transitions (proces sie konczy)

Budowa pojedynczego State

- **Entry** — akcje wykonywane przy wejsciu do stanu (np. pobranie danych, wyswietlenie formularza)
- **Exit** — akcje wykonywane przy wyjsciu ze stanu (np. zapis logow, czyszczenie zmiennych)
- **Transitions** — lista warunkow przejscia do innych stanow. Sprawdzane w kolejnosci zdefiniowania — **pierwsza spelniona wygrywa**.

KIEDY STATE MACHINE ZAMIAST FLOWCHART?

Kryterium	Flowchart	State Machine
Prostota	Prostsze procesy	Zlozone z wieloma stanami
Powtarzanie	Petle przez strzalki	Naturalne przejscia
Czytelnosc	Moze byc chaotyczny przy wielu galeziach	Przejrzysty dla procesow biznesowych
Orchestrator	Wystarczajacy dla prostych	Wzorzec REFramework
Obsluga bledow	Wymaga recznego Try Catch	Naturalnie wbudowana w Transitions

Cwiczenia — Modul 4

Cwiczenie — Input Dialog z rozgalezieniem

Cel: Uzytkownik wpisuje imie. Jesli "Adam" → przypisz nazwisko "Nowak". Jesli "Kamil" → "Kowalski". Inne → ponow pytanie.

CZEGO SIE NAUCZYSZ

- Petla w State Machine — Transition prowadzaca z powrotem do tego samego stanu
- Transition bez warunku — automatyczne przejscie (np. do Final State)
- Kolejnosc Transitions — wazna! Pierwsza spelniona wygrywa
- Scope zmiennych — musza byc na poziomie StateMachine (nie wewnatrz State)
- Final State — nie ma Transitions, konczy State Machine

> **Wskazowka:** To cwiczenie to wstep przed REFramework w Dniu 4. Zauważ analogie: Init → Get Transaction → Process → End.

Przydatne

DEBUGOWANIE — PRZYDATNE TECHNIKI

Flowchart i State Machine

- **Breakpoint w Flowchart:** Kliknij prawym na FlowStep → Toggle Breakpoint. Robot zatrzyma sie przed wykonaniem tego kroku.
- **Sledzenie Transitions w State Machine:** W trybie Debug (F5) UiPath podswietla aktualny State i Transition.
- **Locals Panel:** W trybie Debug panel Locals pokazuje wartosci wszystkich zmiennych — przydatne do sprawdzenia `TransactionItem`, `SystemException` itp.
- **Slow Step:** Wlacz Slow Step w toolbar — spowalnia wykonanie, widoczne kazde przejscie miedzy stanami.

TYPOWE PROBLEMY I ROZWIAZANIA

Problem	Przyczyna	Rozwiazanie
"Variable not found"	Scope zmiennej za maly	Przesun deklaracje zmiennej wyzej
Queue Item pusty	Brak danych w kolejce	Najpierw uruchom Dispatcher
"Queue not found"	Zla nazwa kolejki lub folderu	Sprawdz QueueName i FolderPath w Orchestratorze
Haslo widoczne w logach	Uzycie Text zamiast SecureText	W Type Into uzyj SecureText dla hasla
Plik zablokowany (Excel)	Excel otwarty w tle	Kill Process `excel` przed Read Range
Path Exists zwraca False	Zla sciezka lub PathType	Sprawdz: File vs Folder w PathType